2026-10-24 19:05:29 You

noe write me a project plan from creatingg iam users to testing a sort of logical plan from configuring all my cloud resources , the point i would have and the modification s to make and finally deploying

Gemini 3.1 Pro

This implementation roadmap outlines the sequence to deploy a serverless IoT architecture, moving from local network dependence to global cloud control.

Phase 1: IAM & Environment Setup

- Log into the AWS Management Console and navigate to IAM.
- Create an IAM User named `IoT-Admin` with programmatic access and attach the `AdministratorAccess` policy.
- Generate Access Keys and run `aws configure` in your local terminal to link your PC to your AWS environment.

Phase 2: Database Provisioning

- Open the DynamoDB console and create a table named `DeviceState`.
- Set the Partition Key to `device_id` (Type: String).
- Manually insert a default initialization item: `{"device_id": "esp32_main", "oled_text": "Ready", "led_red": false, "led_yellow": false, "led_green": false}`.

Phase 3: Serverless API Deployment

- Develop a FastAPI Python application with `GET /state/{device_id}` and `POST /state/{device_id}` endpoints that read and update your DynamoDB table using the `boto3` library.
- Wrap the application with the `mangum` adapter to make it compatible with serverless execution.
- Package the dependencies and deploy the code to AWS Lambda.
- Attach an Amazon API Gateway trigger to the Lambda function to generate a public REST endpoint URL.

Phase 4: Remote Dashboard Hosting

- Build a React frontend featuring a text input field and toggle buttons.

- Point the frontend's `fetch()` requests to your new API Gateway URL to push state changes.
- Push the React repository to GitHub and connect it to AWS Amplify for automated continuous deployment and public hosting.

Phase 5: ESP32 Firmware Refactoring

- Strip the local `WebServer` and HTML string logic from your current C++ sketch.
- Import the `HTTPClient` and `ArduinoJson` libraries.
- Write a non-blocking `millis()` timer in the `loop()` to execute an `HTTP GET` request to the API Gateway URL every 3 seconds.
- Parse the JSON response and map the payload to `digitalWrite()` commands for the LEDs and display updates for the OLED.

Phase 6: End-to-End Testing

- Flash the refactored code to the ESP32 while it is connected to your mobile hotspot.
- Open the Amplify web dashboard on a completely separate network (like your PC on home Wi-Fi).
- Send a command via the dashboard. Verify that DynamoDB updates instantly and the ESP32 physically alters its state on the next polling cycle.